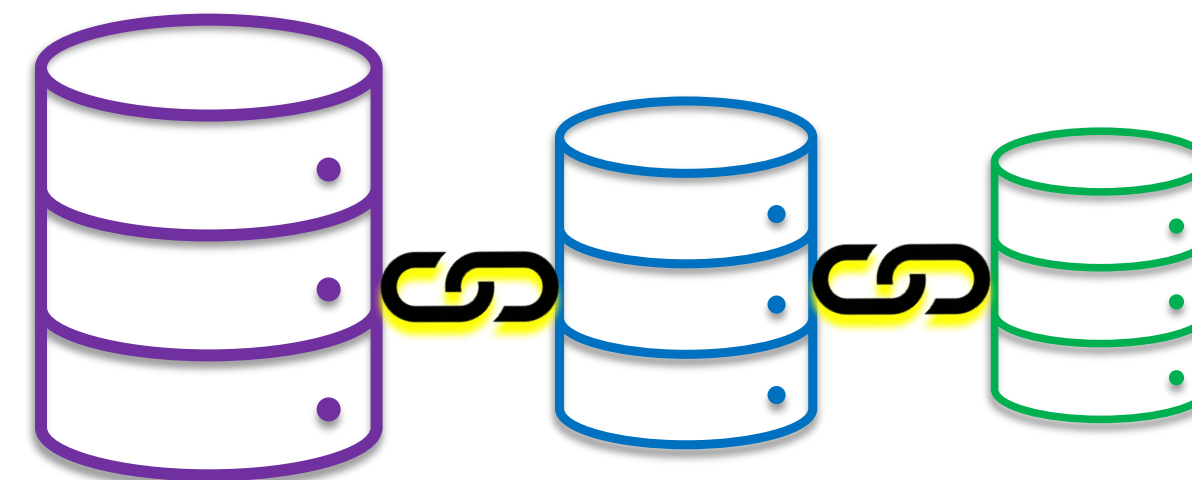
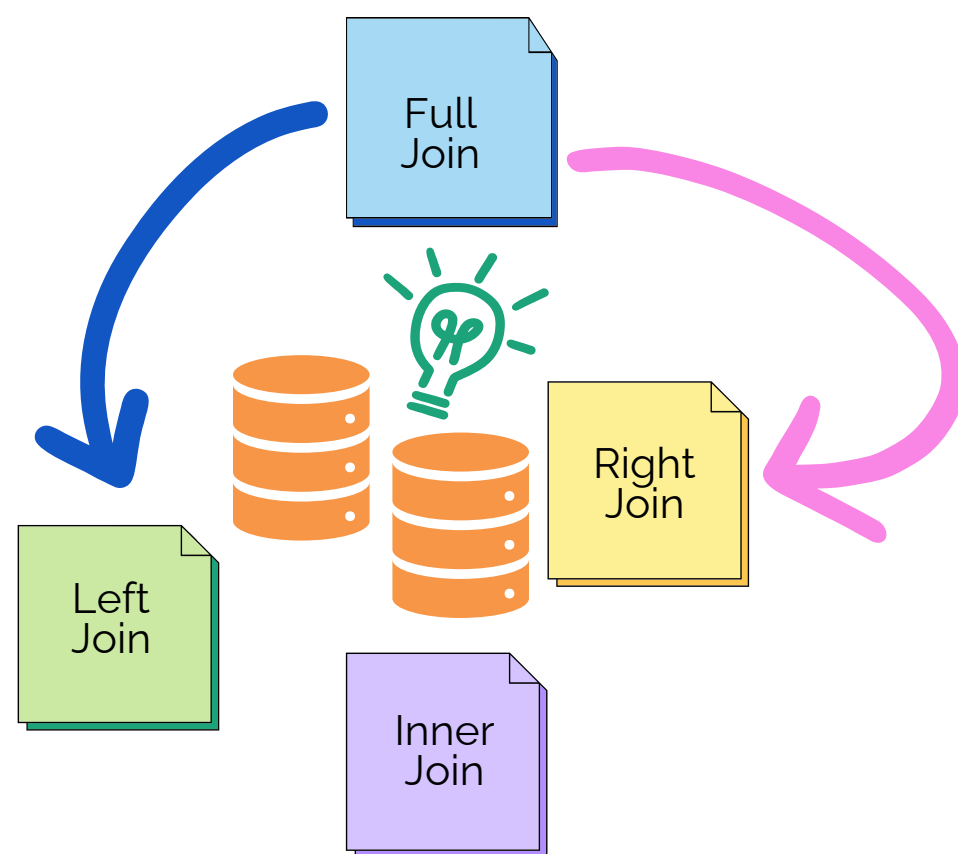


D427 - Data Management Applications

Table Joins & Aggregate Functions



Agenda

①
The Scenario

②
Database Tables

③
Types of Joins

④
SQL Code

⑤
Aggregate Functions



This PPT has some visuals that Zybooks does not and provides another **resource for learning JOINS the Zybooks way.**

zyBooks

To help clarify what you're reading in Zybooks, please open PowerPoint and play it in **Slide Show mode**. That way you can read the notes in the notes section to **teach yourself JOINS**. You'll have visuals to fully understand JOINS and how they work, and **pass the course!**



The Scenario

- You are a **Quantitative Insights Associate** working for a large **Market Research Firm**.
- Your Manager would like you to analyze **orders data from 01/01/2019** (pre-COVID, during COVID lockdowns, return to office) to present day.
- Your DBA (Database Administrator) has **created tables that house the clients' data**.
- **Unique Order IDs and Customer IDs** have been assigned as primary key fields in the Orders and Customers tables.



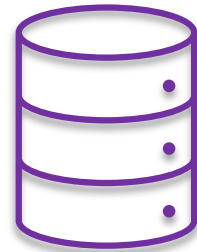
Clients

NETFLIX

hulu



Database Tables



Orders

ALL orders for streaming services and products. Each company has millions of orders, so unique IDs have been assigned.



Customers

ALL customer data, including name, address, email address. Some customers are shared by multiple companies.

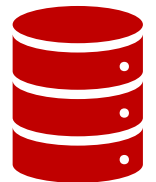


Streaming

This table has the 3 different streaming services (Netflix, Hulu, and Apple TV) at various pricing levels.



Shows



Products



Orders				
●OrderID	○CustID	○StreamID	OrderDate	○ProductID
1	000-00-0001	NET01	1/1/2019	NULL
2	000-00-0003	NET02	2/2/2019	NULL
3	000-00-0003	APL01	10/10/2019	NULL
4	000-00-0004	NULL	4/23/2023	123456
5	000-00-0005	HUL01	7/9/2022	NULL
6	000-00-0006	NET01	3/1/2020	NULL
7	000-00-0007	HUL02	3/15/2021	NULL
8	NULL	NET02	11/12/2021	NULL
9	000-00-0009	APL01	8/6/2023	NULL

Customers					
●CustID	CFName	CLName	Address	City	State
000-00-0001	Kevin	Hart	123 1st St.	Philadelphia	PA
000-00-0002	Amy	Pohler	456 2nd St.	Newton	MA
000-00-0003	Chris	Rock	789 3rd St.	Andrews	SC
000-00-0004	Ellen	Degeneres	901 4th St.	Metairie	LA
000-00-0005	Wanda	Sykes	234 5th St.	Portsmouth	VA
000-00-0006	Gabriel	Iglesias	567 6th St.	San Diego	CA
000-00-0007	Jerry	Seinfeld	890 7th St.	New York	NY
000-00-0008	George	Lopez	123 8th St.	Los Angeles	CA
000-00-0009	Eddie	Murphy	456 9th St.	New York	NY

Streaming		
●StreamID	StreamName	Price
NET01	Netflix With Ads	\$ 6.99
NET02	Netflix No Ads	\$15.49
APL01	Apple TV	\$ 6.99
HUL01	Hulu With Ads	\$ 7.99
HUL02	Hulu No Ads	\$14.99

The FIRST TABLE the question mentions or the table on TOP will ALWAYS be OUR LEFT Table. The second table mentioned, or the table below will be our RIGHT Table.

● PRIMARY KEY
○ FOREIGN KEY

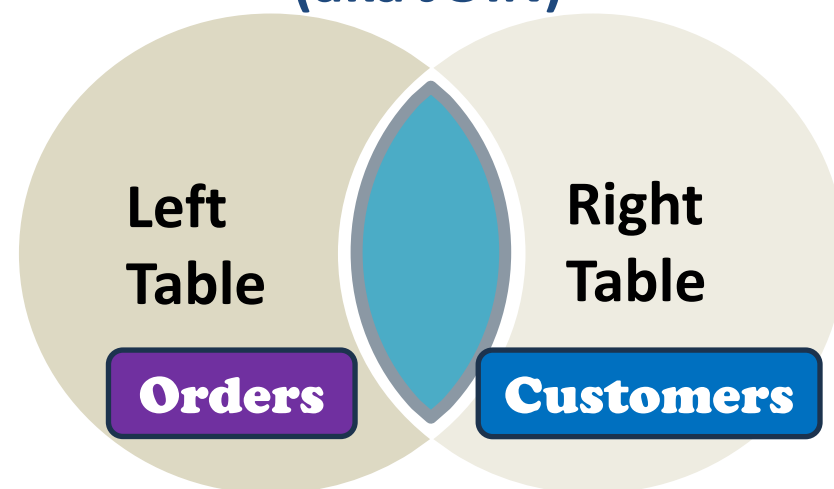


Types of Joins

A **JOIN** is an operation in SQL in which **tables are compared to each other row by row**. The comparison allows you to output columns from any table involved in the join. The **first table** mentioned will ALWAYS be our **LEFT table**.

You know it is an **INNER JOIN** when there are **NO STIPULATIONS** in the question.

INNER JOIN (aka JOIN)

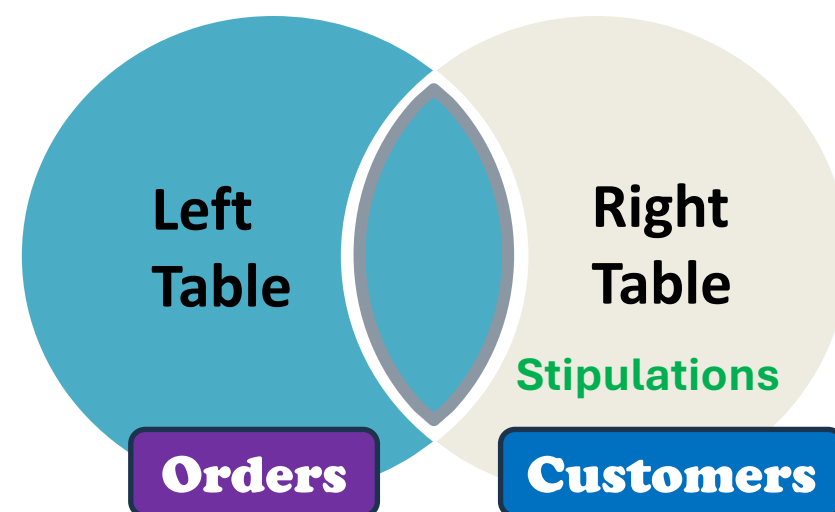


Returns a dataset that has **any matching** values in **both** tables.



In the OA and Pre-A every **LEFT or RIGHT JOIN** has **STIPULATIONS** written in the question such as: **(if available, regardless of, whether or not, even if)**.

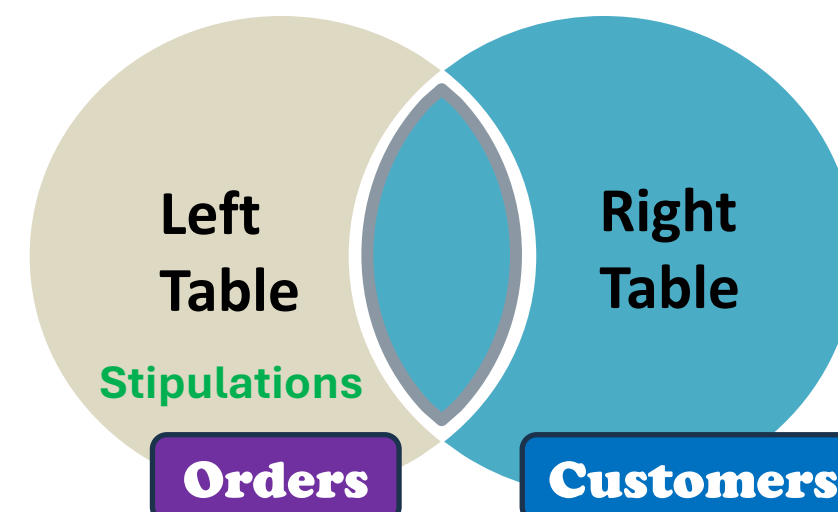
LEFT JOIN - Outer



Returns **all records** from the **LEFT table** and the **matching records that meet the stipulation** from the right table.

If there is **NOT a match in the Right Table**, the record will **populate with NULL**.

RIGHT JOIN - Outer

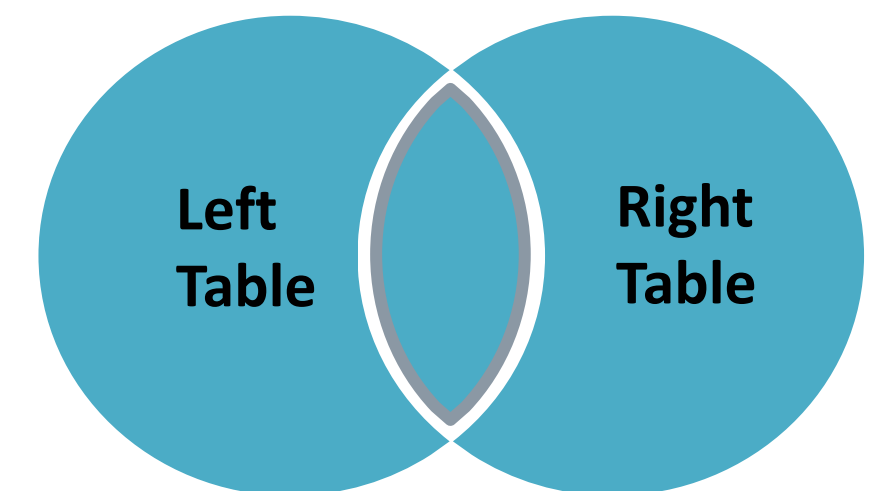


Returns **ALL records** from the **RIGHT table** and the **matching records that meet the stipulation** from the left table.

If there is **NOT a match in the Left Table**, the record will **populate with NULL**.

FULL JOINS are rarely used in practical applications.

FULL JOIN - Outer



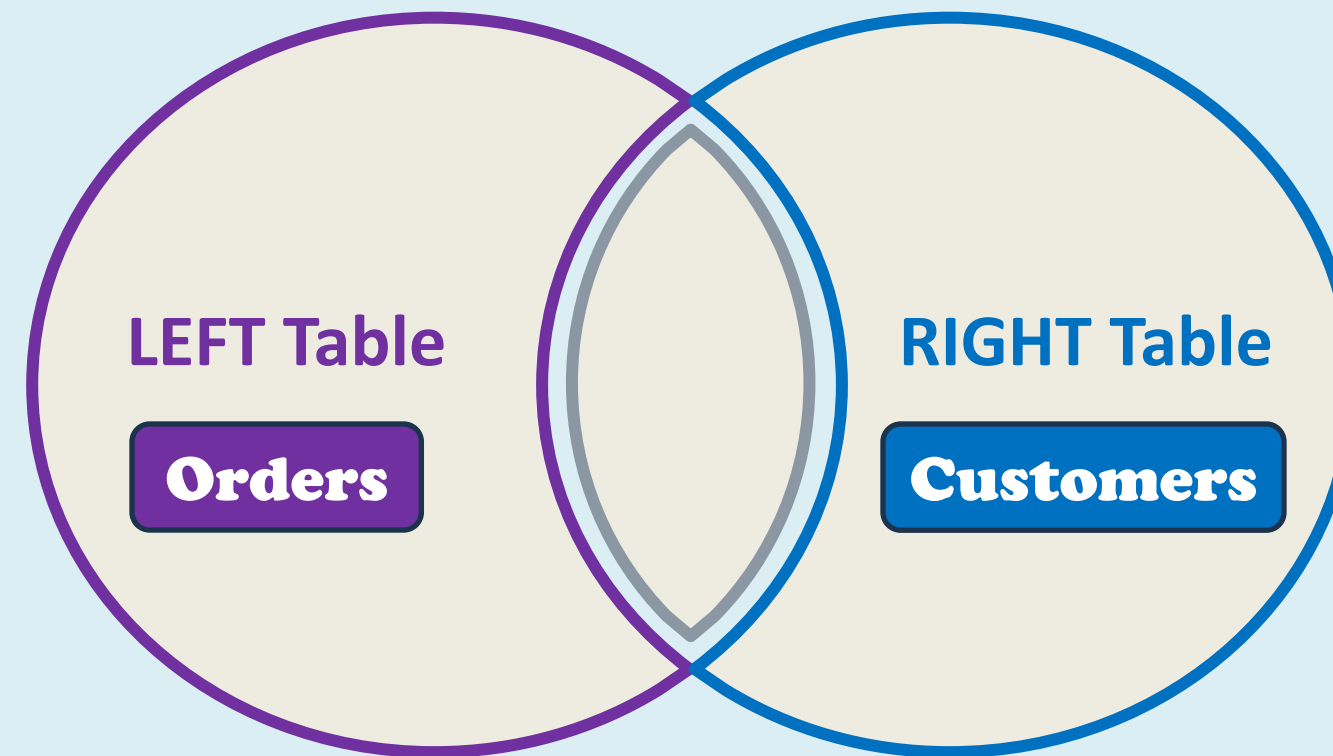
Returns **all records** that includes rows from **both left and right tables**. If a row from one table does not have a corresponding record from the other table, then the missing data is filled with **NULLs**.



SQL CODE FOR JOINS



The easiest way to write JOINS is to **always JOIN from the LEFT table to the RIGHT table**, no matter which JOIN (LEFT/RIGHT/INNER) you are using. The LEFT Table is always the FIRST table shown in the question.



Make sure to return the columns in the order indicated in the question.

```
SELECT Tablename.ColumnName, Tablename.ColumnName  
FROM LEFTTablename
```

```
LEFT|RIGHT|INNER JOIN RIGHTTablename  
STIPULATIONS
```

```
ON LEFTTablename.ColumnName = RIGHTTablename.ColumnName;
```



The Column that is identical in both tables.

FOREIGN KEY = PRIMARY KEY/FOREIGN KEY

Orders				
●OrderID	○CustID	○StreamID	OrderDate	○ProductID
1	000-00-0001	NET01	1/1/2019	NULL
2	000-00-0003	NET02	2/2/2019	NULL
3	000-00-0003	APL01	10/10/2019	NULL
4	000-00-0004	NULL	4/23/2023	123456
5	000-00-0005	HUL01	7/9/2022	NULL

JOINS EXAMPLE – 3 STEPS

Customers					
●CustID	CFName	CLName	Address	City	State
000-00-0001	Kevin	Hart	123 1st St.	Philadelphia	PA
000-00-0002	Amy	Pohler	456 2nd St.	Newton	MA
000-00-0003	Chris	Rock	789 3rd St.	Andrews	SC
000-00-0004	Ellen	Degeneres	901 4th St.	Metairie	LA
000-00-0005	Wanda	Sykes	234 5th St.	Portsmouth	VA

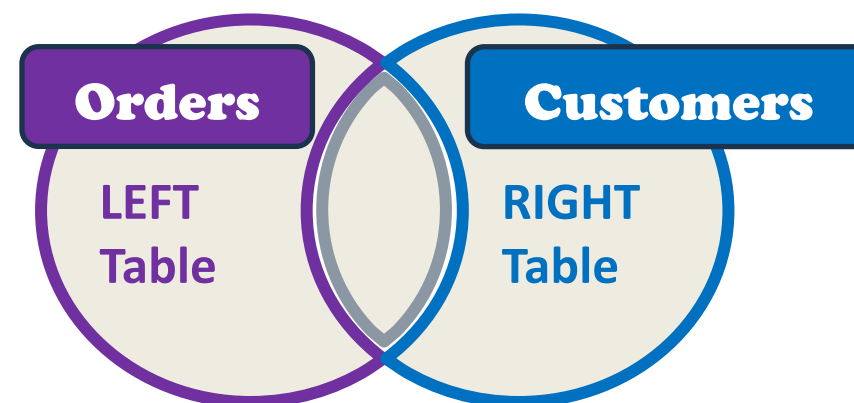
Zybooks Question: Using the Orders table and Customers table, display the streaming ID and customer's first name if they placed a streaming order, for all customers.

- 1) The question lists two tables and is asking you to display/select/pull data from both tables... that must be a JOIN! **Start by typing the first four lines of code:**

```
SELECT
FROM
JOIN
ON
```

- 2) Now decide which table is on the left. The FIRST table or TOP table in the question will be your **LEFT table**, and the 2nd one will be your **RIGHT table**.

```
SELECT
FROM ORDERS
JOIN CUSTOMERS
ON
```



- 3) You re-read the question and **look for STIPULATIONS** (i.e. 'if available', 'regardless of', 'whether or not', 'even if there are nulls').

IF there is **NO STIPULATION** it is an **INNER JOIN**.

IF there **IS a STIPULATION**, it **MUST** be:
a **LEFT or RIGHT JOIN**.

This question says 'streaming ID **IF** they placed a streaming order' **IF = STIPULATION** (on the Orders table, so bring **SOME** of the streaming orders) **for ALL customers (ALL Customer First Names from the RIGHT table)**, which makes this a **RIGHT JOIN**.

```
SELECT Orders.StreamID, Customers.CFName
FROM Orders
RIGHT JOIN Customers
ON Orders.CustID = Customers.CustID;
```

The Column that is identical in both tables.

Make sure to return the columns in the order indicated in the question.

JOINS Video – Courtesy of Marissa (a former student)

Screenshot 2024-03-15 134000.png

www.BANDICAM.com

Save options Cancel

FINAL ANSWER

```
SELECT Movie.Title, YearStats.TotalGross
FROM Movie
LEFT JOIN YearStats
ON Movie.Year = YearStats.Year;
```

The **Movie** table has the following columns:

- **ID** - integer, primary key
- **Title** - variable-length string
- **Genre** - variable-length string
- **RatingCode** - variable-length string
- **Year** - integer

The **YearStats** table has the following columns:

- **Year** - integer
- **TotalGross** - bigint unsigned
- **Releases** - integer

Write a SQL query to display both the **Title** and the **TotalGross** (if available) for all movies. Ensure your result set returns the columns in the order indicated.

Joins - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

Joins

```
1 Joins
2
3 Simple Version - Structure
4
5
6
7 ----
8
9 Detailed Version - Structure
10
11
12
13
```

Line 5, Column 1

Tab Size: 4 Plain Text

INNER JOIN

LEFT JOIN

RIGHT JOIN

104.5%

JOIN Tables

Orders

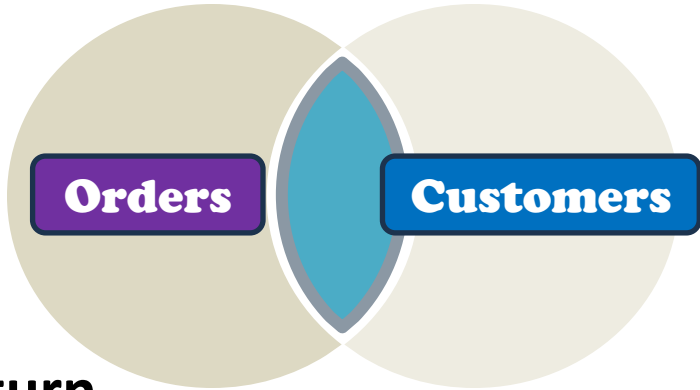
OrderID	CustID	StreamID	OrderDate	ProdID
1	000-00-0001	NET01	1/1/2019	NULL
2	000-00-0003	NET02	2/2/2019	NULL
3	000-00-0003	APL01	10/10/2019	NULL
4	000-00-0004	NULL	4/23/2023	123456
5	000-00-0005	HUL01	7/9/2022	NULL
6	000-00-0006	NET01	3/1/2020	NULL
7	000-00-0007	HUL02	3/15/2021	NULL
8	NULL	NET02	11/12/2021	NULL
9	000-00-0009	APL01	8/6/2023	NULL

Customers

CustID	CFName	CLName	Address	City	State
000-00-0001	Kevin	Hart	123 1st St.	Philadelphia	PA
000-00-0002	Amy	Pohler	456 2nd St.	Newton	MA
000-00-0003	Chris	Rock	789 3rd St.	Andrews	SC
000-00-0004	Ellen	Degeneres	901 4th St.	Metairie	LA
000-00-0005	Wanda	Sykes	234 5th St.	Portsmouth	VA
000-00-0006	Gabriel	Iglesias	567 6th St.	San Diego	CA
000-00-0007	Jerry	Seinfeld	890 7th St.	New York	NY
000-00-0008	George	Lopez	123 8th St.	Los Angeles	CA
000-00-0009	Eddie	Murphy	456 9th St.	New York	NY

**Orders data is from 01/01/2019, but Customer data includes all customers even those who placed orders before 01/01/2019.

SQL Code - Inner JOIN



1st Data Pull

Pull out any orders with matching customer information and return the Streaming ID, Customer First Name and Customer Last Name.

ZyBooks Way: Write a query to show the Streaming ID and Customer First and Last Name for all orders.

OR

Zybooks Way: Write a query to display the Streaming ID and Customer First and Last Name.

NO Stipulation in the question.

SQL Code

```
SELECT Orders.StreamID, Customers.CFName, Customers.CLName
FROM Orders
INNER JOIN Customers
ON Orders.CustID = Customers.CustID;
```

Run SQL >>

StreamID	CFName	CLName
NET01	Kevin	Hart
NET02	Chris	Rock
APL01	Chris	Rock
NULL	Ellen	Degeneres
HUL01	Wanda	Sykes
NET01	Gabriel	Iglesias
HUL02	Jerry	Seinfeld
APL01	Eddie	Murphy

Which records are missing from the inner join?

Anytime you JOIN 2 or more tables, use this format!

8 Records ONLY Matching CustIDs

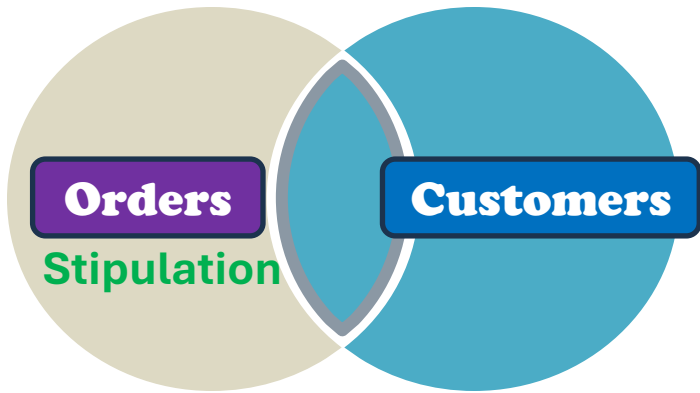
JOIN Tables

Orders				
OrderID	CustID	StreamID	OrderDate	ProdID
1	000-00-0001	NET01	1/1/2019	NULL
2	000-00-0003	NET02	2/2/2019	NULL
3	000-00-0003	APL01	10/10/2019	NULL
4	000-00-0004	NULL	4/23/2023	123456
5	000-00-0005	HUL01	7/9/2022	NULL
6	000-00-0006	NET01	3/1/2020	NULL
7	000-00-0007	HUL02	3/15/2021	NULL
8	NULL	NET02	11/12/2021	NULL
9	000-00-0009	APL01	8/6/2023	NULL

Customers					
CustID	CFName	CLName	Address	City	State
000-00-0001	Kevin	Hart	123 1st St.	Philadelphia	PA
000-00-0002	Amy	Pohler	456 2nd St.	Newton	MA
000-00-0003	Chris	Rock	789 3rd St.	Andrews	SC
000-00-0004	Ellen	Degeneres	901 4th St.	Metairie	LA
000-00-0005	Wanda	Sykes	234 5th St.	Portsmouth	VA
000-00-0006	Gabriel	Iglesias	567 6th St.	San Diego	CA
000-00-0007	Jerry	Seinfeld	890 7th St.	New York	NY
000-00-0008	George	Lopez	123 8th St.	Los Angeles	CA
000-00-0009	Eddie	Murphy	456 9th St.	New York	NY

**Orders data is from 01/01/2019, but Customer data includes all customers even those who placed orders before 01/01/2019.

SQL Code – Right JOIN



3rd Data Pull

ZyBooks Way: Write a query to display the Streaming ID **if available**, and Customer First and Last Name, for all customers. **Display the data alphabetically by customer first name.**

SQL Code

```
SELECT Orders.StreamID, Customers.CFName, Customers.CLName
FROM Orders
RIGHT JOIN Customers
ON Orders.CustID = Customers.CustID
ORDER BY Customers.CFName ASC;
```



Run SQL >>

StreamID	CFName	CLName
NULL	Amy	Pohler
NET02	Chris	Rock
APL01	Chris	Rock
APL01	Eddie	Murphy
NULL	Ellen	Degeneres
NET01	Gabriel	Iglesias
NULL	George	Lopez
HUL02	Jerry	Seinfeld
NET01	Eddie	Murphy
HUL01	Wanda	Sykes

10 Records
ALL Customers +
matching Orders

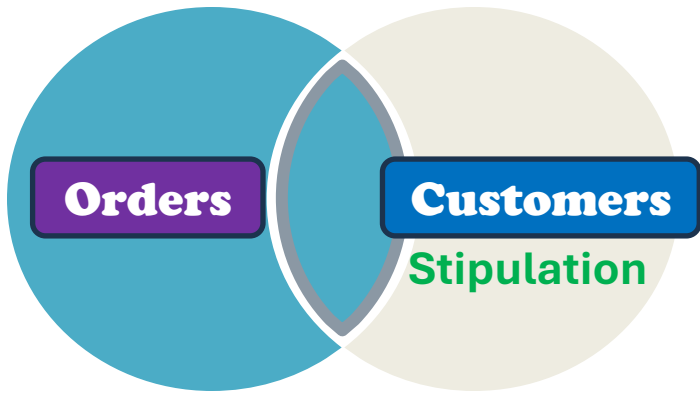
Which records is missing from the RIGHT join?

JOIN Tables

Orders				
OrderID	CustID	StreamID	OrderDate	ProdID
1	000-00-0001	NET01	1/1/2019	NULL
2	000-00-0003	NET02	2/2/2019	NULL
3	000-00-0003	APL01	10/10/2019	NULL
4	000-00-0004	NULL	4/23/2023	123456
5	000-00-0005	HUL01	7/9/2022	NULL
6	000-00-0006	NET01	3/1/2020	NULL
7	000-00-0007	HUL02	3/15/2021	NULL
8	NULL	NET02	11/12/2021	NULL
9	000-00-0009	APL01	8/6/2023	NULL

Customers					
CustID	CFName	CLName	Address	City	State
000-00-0001	Kevin	Hart	123 1st St.	Philadelphia	PA
000-00-0002	Amy	Pohler	456 2nd St.	Newton	MA
000-00-0003	Chris	Rock	789 3rd St.	Andrews	SC
000-00-0004	Ellen	Degeneres	901 4th St.	Metairie	LA
000-00-0005	Wanda	Sykes	234 5th St.	Portsmouth	VA
000-00-0006	Gabriel	Iglesias	567 6th St.	San Diego	CA
000-00-0007	Jerry	Seinfeld	890 7th St.	New York	NY
000-00-0008	George	Lopez	123 8th St.	Los Angeles	CA
000-00-0009	Eddie	Murphy	456 9th St.	New York	NY

SQL Code - Left JOIN



2nd Data Pull

ZyBooks Way: Display the Streaming ID and Customer First and Last Name **whether or not** there is a customer first or last name for all orders.

SQL Code

```
SELECT Orders.StreamID, Customers.CFName, Customers.CLName
FROM Orders
LEFT JOIN Customers
ON Orders.CustID = Customers.CustID;
```

Run SQL >>

StreamID	CFName	CLName
NET01	Kevin	Hart
NET02	Chris	Rock
APL01	Chris	Rock
NULL	Ellen	Degeneres
HUL01	Wanda	Sykes
NET01	Gabriel	Iglesias
HUL02	Jerry	Seinfeld
NET02	NULL	NULL
APL01	Eddie	Murphy

9 Records
ALL Orders +
matching Customers

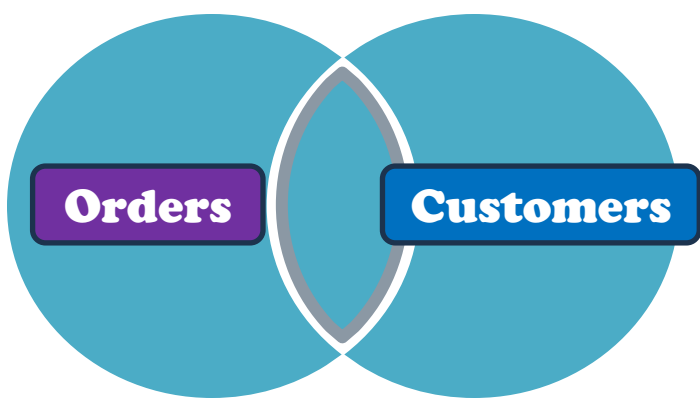
Which records are missing from the LEFT join?

JOIN Tables

Orders				
OrderID	CustID	StreamID	OrderDate	ProdID
1	000-00-0001	NET01	1/1/2019	NULL
2	000-00-0003	NET02	2/2/2019	NULL
3	000-00-0003	APL01	10/10/2019	NULL
4	000-00-0004	NULL	4/23/2023	123456
5	000-00-0005	HUL01	7/9/2022	NULL
6	000-00-0006	NET01	3/1/2020	NULL
7	000-00-0007	HUL02	3/15/2021	NULL
8	NULL	NET02	11/12/2021	NULL
9	000-00-0009	APL01	8/6/2023	NULL

Customers					
CustID	CFName	CLName	Address	City	State
000-00-0001	Kevin	Hart	123 1st St.	Philadelphia	PA
000-00-0002	Amy	Pohler	456 2nd St.	Newton	MA
000-00-0003	Chris	Rock	789 3rd St.	Andrews	SC
000-00-0004	Ellen	Degeneres	901 4th St.	Metairie	LA
000-00-0005	Wanda	Sykes	234 5th St.	Portsmouth	VA
000-00-0006	Gabriel	Iglesias	567 6th St.	San Diego	CA
000-00-0007	Jerry	Seinfeld	890 7th St.	New York	NY
000-00-0008	George	Lopez	123 8th St.	Los Angeles	CA
000-00-0009	Eddie	Murphy	456 9th St.	New York	NY

SQL Code - Full JOIN



4th Data Pull

Display all orders and all customers and return the Streaming ID, Customer First and Last Name, and Product ID. Sort the data alphabetically by first name.

In D427 V3, you will have to write a UNION of a LEFT and RIGHT JOIN, since the FULL JOIN is not fully supported by MySQL. FULL JOINS are NOT in D427 V2.

SQL Code

```
SELECT Orders.StreamID, Customers.CFName, Customers.CLName, Orders.ProdID
FROM Orders
FULL JOIN Customers
ON Orders.CustID = Customers.CustID
ORDER BY Customers.CFName ASC;
```

Run SQL >>

StreamID	CFName	CLName	ProdID
NULL	Amy	Pohler	NULL
NET02	Chris	Rock	NULL
APL01	Chris	Rock	NULL
APL01	Eddie	Murphy	NULL
NULL	Ellen	Degeneres	123456
NET01	Gabriel	Iglesias	NULL
NULL	George	Lopez	NULL
HUL02	Jerry	Seinfeld	NULL
NET01	Kevin	Hart	NULL
NET02	NULL	NULL	NULL
HUL01	Wanda	Sykes	NULL

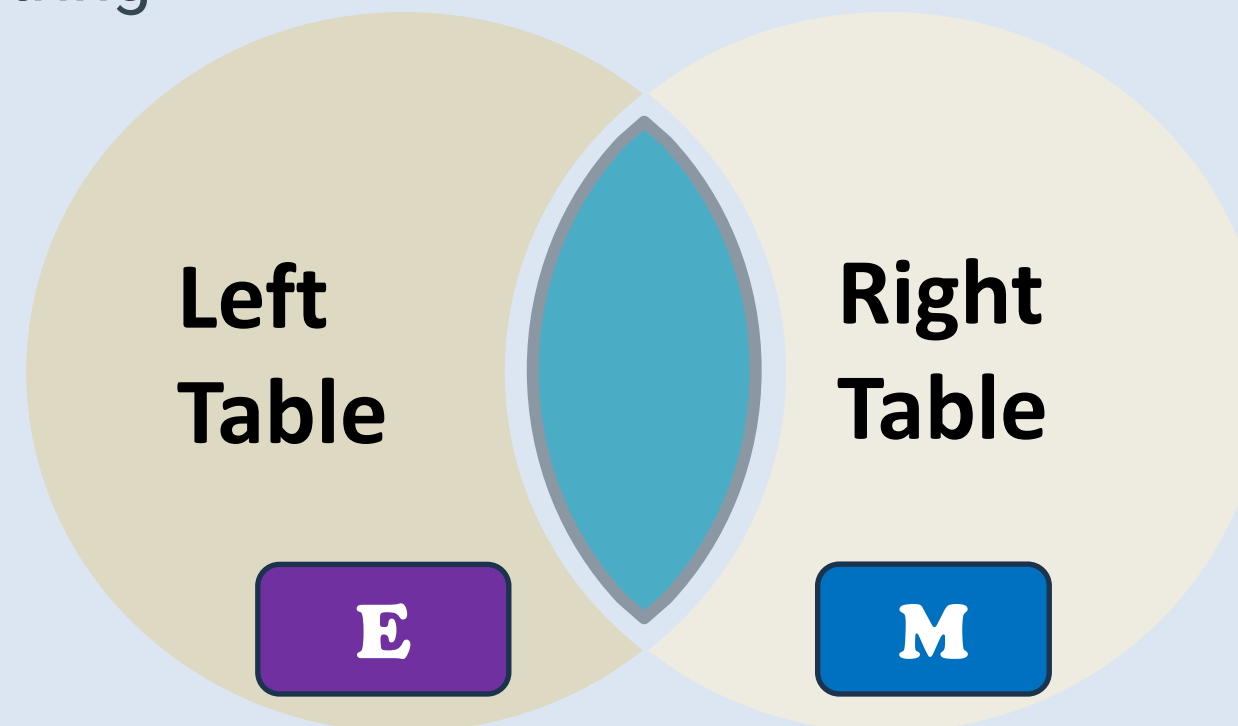
11 Records
ALL DATA in
both tables

EVERY record is included in the FULL join.

SELF-JOIN

The **Employee** table has the following columns:

- **ID** - integer, primary key
- **FirstName** - variable-length string
- **LastName** - variable-length string
- **ManagerID** - integer



```
SELECT E.FirstName AS Employee, M.Firstname AS Manager
FROM Employee E
INNER JOIN Employee M
ON E.ManagerID = M.ID
ORDER BY E.Firstname;
```

A SELF-JOIN is used when you JOIN a copy of a table to itself to compare rows of the same table.

You must give each table in a SELF-JOIN an Alias (usually a letter to represent the copy of the table).

It is used to display a hierarchy of data.

Only use ALIASES on the OA if the question asks for an ALIAS.

Aggregate Functions



Aggregate Functions are mathematical functions. Think of them as a **calculation you perform by passing a column through** the function which returns a single value.

MIN(ColumnName), **MAX**(ColumnName),
AVG(ColumnName), **SUM**(ColumnName),
COUNT(*)



If the question asks you to pull the **unique EmployeeType** and the **highest salary by EmployeeType** in the Employee table, you will use an aggregate function.

```
SELECT DISTINCT EmployeeType, MAX(Salary)
FROM Employee
GROUP BY EmployeeType;
```

Aggregate Functions

Streaming

Shows



The **Shows** table has the following columns:

ShowID—integer, primary key

Title—variable-length string

TargetAudience—variable-length string

ReleaseDate – date

Cost – integer, positive

StreamID— fixed length character

1) Write a SQL statement to pull the average show cost.

```
SELECT AVG(Cost)
FROM Shows;
```

2) Write a SQL statement to pull all data for shows that were **above average cost** to make.

```
SELECT *
FROM Shows
WHERE Cost > (SELECT AVG(Cost) FROM Shows);
```

Requires a **(SubQuery)** because of the WHERE clause.

3) Write a SQL statement to **count all shows** for the **Millennials** target audience.

```
SELECT COUNT(*)
FROM Shows
WHERE TargetAudience = 'Millennials';
```

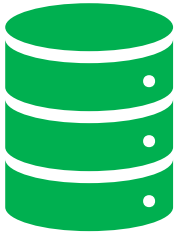
4) Write a SQL statement to pull the **show that cost the least** in the Shows Table.

```
SELECT MIN(Cost)
FROM Shows;
```

Aggregate Functions

Streaming

Shows



The **Shows** table has the following columns:
ShowID—integer, primary key
Title—variable-length string
TargetAudience—variable-length string
ReleaseDate - date
StreamID— fixed length character

Write a SQL query to output the **unique** TargetAudience values and the number of shows by TargetAudience from the Shows table as TargetAudienceCount. **Sort/Display** the results by the TargetAudience in reverse alphabetical order Z-A.

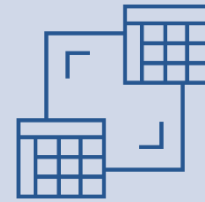
```
SELECT DISTINCT TargetAudience, COUNT(*) AS TargetAudienceCount
FROM Shows
GROUP BY TargetAudience
ORDER BY TargetAudience DESC;
```

We use **GROUP BY** to **aggregate data** into categories, and **ORDER BY to sort** the data in a specific order i.e., ASC(lowest to highest or A-Z) or DESC(highest to lowest or Z-A).

TargetAudience	TargetAudienceCount
Young Children	86
Toddlers	55
Teens	45
Silent-Generation	12
Millennials	120
Gen Z	45
Gen X	130
Baby Boomers	70



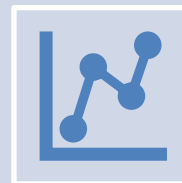
Summary



JOINS are useful when you are **pulling data from multiple tables**.



We use JOIN clauses to **extract specific data** from tables using SQL.



Understanding the differences between JOINS, will **ensure** you extract the specific data that you need for analysis.